04 MAR 2026

AADK TASK 2

Large-Scale UI Feature Plan

MindMatrix

Feature Description

A Superheroes Directory app feature that displays 50+ heroes with image, name, alias, and power summary. Users can search by name/power, filter favourites, view hero details, and toggle favourite status. The UI will be accessible and fully tested.

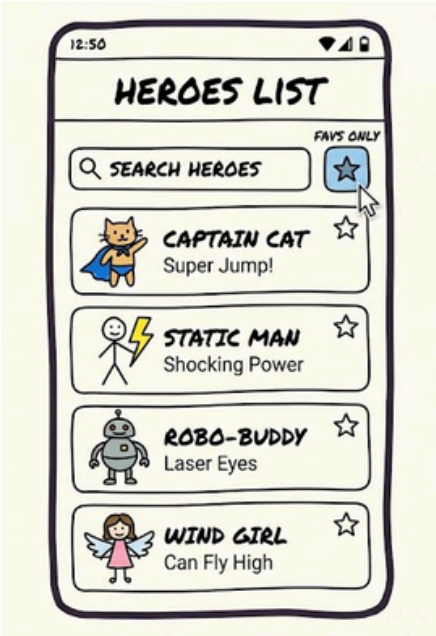
Data Model

```
data class Superhero(
    val id: Int,
    val name: String,
    val alias: String?,
    val powerSummary: String,
    val imageUrl: String,
    val isFavourite: Boolean
```

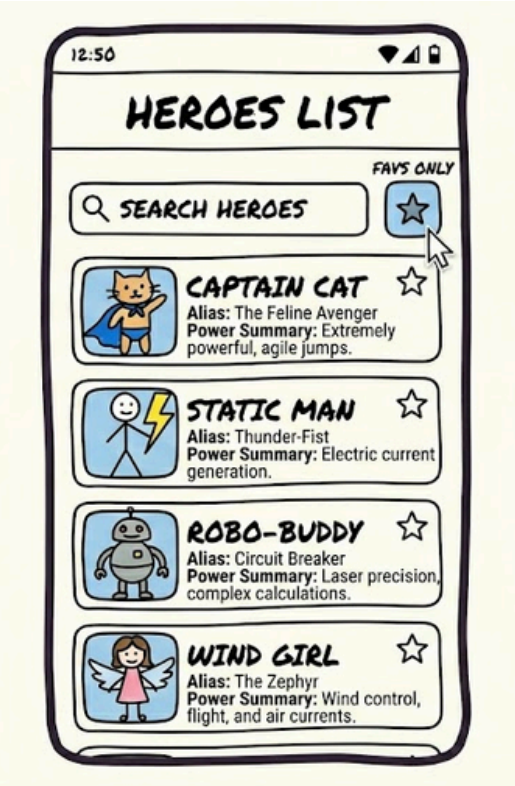
Sample dataset: 10+ heroes (Spider-Man, Iron Man, Thor, Batman, Wonder Woman, etc.) with local/URL images.)

UI Layout Plan

Heroes List Screen



Each Card:



Hero Detail Screen

State & Logic

Main states:

- searchQuery
- filterFavourite
- heroesList
- selectedHero

Filtered List Logic:

- Filter by search text (name/power)
- If favourite filter ON → show only favourites
-

Favourite toggle updates hero state → UI recomposes automatically.

Accessibility & Testing

Accessibility:

- Image: "${hero.name} photo"
- Favourite button labelled
- Search bar labelled
- Touch targets ≥ 48dp
- Supports font scaling

Automated Tests:

- Search filters correctly
- Favourite toggle updates UI
- Detail screen shows correct data
- Images have contentDescription
- State preserved on rotation

Performance Considerations

- Use LazyColumn
- Load images with Coil (cached + placeholder)
- Use stable keys (hero.id)
- Plan pagination if dataset grows beyond 50+

Reflection

This feature is more complex because it includes navigation, filtering, state management, favourites, and accessibility together. Search and filter logic increase UI state complexity. Planning accessibility and tests early helps make the UI more structured, scalable, and reliable.

6. Reflection

Designing animation before coding helps in planning smoother user interactions and avoiding unnecessary complexity. It ensures that animations serve a purpose instead of being randomly added. When animation is well planned, the app feels polished and professional.

Overusing animation can make the app distracting and slow. Too much motion may also confuse users or reduce usability. I will decide not to animate when the action is frequent (like scrolling) or when animation does not add meaningful feedback.

Animations should enhance clarity — not just look fancy.